

Laboratorio práctico de Docker — Parte II: **docker-compose**

Obxectivo xeral: Orquestrar servizos incrementando complexidade: variables de entorno, *depends_on* con *health*, volumes, redes, perfís, secrets e escalado.

Requisitos previos

- Docker Compose v2.
- Reutiliza a carpeta **docker-lab/** con subdirectorios por exercicio.

Estrutura suxerida:

```
compose/  
├── 01-basic/  
├── 02-env-depends/  
├── 03-postgres/  
├── 04-net-profiles/  
├── 05-configs-secrets/  
└── 06-scale-logs/
```

A) O básico: Nginx servindo HTML local

Que vas facer: un servizo con porto publicado e *bind mount* de HTML estático.

Estrutura (**compose/01-basic/**):

```
compose/01-basic/  
├── docker-compose.yml  
└── html/  
    └── index.html
```

docker-compose.yml:

```
services:  
  web:  
    image: nginx:1.27-alpine  
    ports:  
      - "8080:80"  
    volumes:  
      - ./html:/usr/share/nginx/html:ro
```

html/index.html:

```
<!doctype html>
<html>
  <body>
    <h1>Nginx en Compose</h1>
  </body>
</html>
```

Proba:

```
cd compose/01-basic
docker compose up
# http://localhost:8080
```

Puntos clave: rutas relativas, **ports** vs **expose**.

Criterios de éxito: páxina servida.

B) Env vars, healthcheck e **depends_on**

Que vas facer: API Flask + Redis con dependencia por *health* e configuración vía **environment**.

Estrutura (**compose/02-env-depends/**):

```
compose/02-env-depends/
├── docker-compose.yml
├── api/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
```

api/requirements.txt:

```
Flask==3.0.3
redis==5.0.8
```

api/app.py:

```
import os, redis
from flask import Flask, jsonify

r = redis.Redis(host=os.getenv("REDIS_HOST","redis"), port=6379)
app = Flask(__name__)

@app.get("/")
```

```
def ix():
    r.incr("hits")
    return jsonify(hits=int(r.get("hits") or 0))

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

api/Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

docker-compose.yml:

```
services:
  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  api:
    build: ./api
    environment:
      - REDIS_HOST=redis
      - FLASK_ENV=production
    ports:
      - "5000:5000"
    depends_on:
      redis:
        condition: service_healthy
```

Proba:

```
cd compose/02-env-depends
docker compose up --build
# http://localhost:5000 → {"hits":1}
```

Puntos clave: `depends_on` con condiciones, env en runtime vs build args.

Criterios de éxito: API arrinca cando Redis está *healthy* e incrementa `hits`.

C) Volumes nomeados, datos persistentes e `.env`

Que vas facer: Postgres con persistencia en volume e `POSTGRES_PASSWORD` nun `.env`.

Estrutura (`compose/03-postgres/`):

```
compose/03-postgres/  
├─ docker-compose.yml  
└─ .env
```

`.env`:

```
POSTGRES_PASSWORD=supersegredo
```

`docker-compose.yml`:

```
services:  
  db:  
    image: postgres:16-alpine  
    ports:  
      - "5432:5432"  
    environment:  
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}  
    volumes:  
      - pgdata:/var/lib/postgresql/data  
  
  client:  
    image: postgres:16-alpine      # usa a mesma imaxe, xa trae psql  
    depends_on:  
      - db  
    entrypoint: ["psql"]  
    command:  
      - "-h"  
      - "db"  
      - "-U"  
      - "postgres"  
    environment:  
      - PGPASSWORD=${POSTGRES_PASSWORD}  
    tty: true                      # permite abrir terminal interactivo  
    stdin_open: true              # necesario para sesión manual  
  
volumes:  
  pgdata:
```

Proba:

```
cd compose/03-postgres
docker compose --env-file .env up -d
# conectar con cliente psql/gui → datos persisten tras recreación
docker compose run --rm client
```

Podemos crear unha táboa de proba, meter datos, deter o compose e despois comprobar que os datos persisten:

```
-- 1. Crear a táboa
CREATE TABLE estudantes (
  id INT PRIMARY KEY,
  nome VARCHAR(50),
  idade INT,
  curso VARCHAR(50)
);

-- 2. Inserir algúns datos
INSERT INTO estudantes (id, nome, idade, curso) VALUES
(1, 'Ana López', 20, 'Big Data'),
(2, 'Carlos Pérez', 22, 'Ciberseguridade'),
(3, 'Marta García', 21, 'Desenvolvemento de Aplicacións Web');

-- 3. Consultar os datos
SELECT * FROM estudantes;
```

Puntos clave: diferenza volume nomeado vs bind mount, backups.

Criterios de éxito: datos persisten tras **down** e nova subida.

D) Redes, perfís e restart

Que vas facer: illar comunicación en redes separadas, activar servizos opcionais con *profiles* e configurar **restart**.

Estrutura (`compose/04-net-profiles/`):

```
compose/04-net-profiles/
└─ docker-compose.yml
```

`docker-compose.yml`:

```
networks:
  frontend: {}
  backend: {}

services:
  web:
    image: nginx:1.27-alpine
    ports: ["8080:80"]
    networks: [frontend]
    restart: unless-stopped

  api:
    build: ../02-env-depends/api
    networks: [frontend, backend]

  redis:
    image: redis:7-alpine
    networks: [backend]
    profiles: [cache]
```

Probas:

```
cd compose/04-net-profiles
docker compose up --build
docker compose --profile cache up -d redis
docker network inspect compose04-net-profiles_frontend
docker network inspect compose04-net-profiles_backend
```

Puntos clave: service discovery por nome, perfís para dev/test.

Criterios de éxito: illamento correcto e arranque selectivo con perfís.

E) Configs, secrets e límites locais

Que vas facer: montar configuración en *bind mount*, xestionar secrets en Compose (modo local) e comentar límites de recursos.

Estrutura (`compose/05-configs-secrets/`):

```
compose/05-configs-secrets/
├── docker-compose.yml
├── config/
│   └── nginx.conf
├── secrets/
│   └── api_key.txt  # non subir a repo
```

`docker-compose.yml`:

```

services:
  web:
    image: nginx:1.27-alpine
    ports: ["8080:80"]
    volumes:
      - ./config/nginx.conf:/etc/nginx/nginx.conf:ro
    environment:
      - API_KEY_FILE=/run/secrets/api_key
    secrets:
      - api_key

secrets:
  api_key:
    file: ./secrets/api_key.txt

```

config/nginx.conf

```

# user nginx; # opcional (ou elimina a directiva)
worker_processes 1;

events { worker_connections 1024; }

http {
    # Sen formato de log personalizado para evitar 'main'
    access_log /var/log/nginx/access.log;

    server {
        listen 8080;
        server_name _;

        root /usr/share/nginx/html;
        index index.html;

        location /health { return 200 'ok'; }

        location / { try_files $uri $uri/ =404; }
    }
}

```

Nota: `deploy.resources` aplica en Swarm; en local, límites finos aplícanse vía CLI/daemon.

Criterios de éxito: Nginx arrinca coa configuración montada e o segredo é accesible dentro do contedor.

F) Escalado e logs

Que vas facer: escalar un servizo e observar logs das réplicas.

Probas (reuse de B):

```
cd compose/02-env-depends
docker compose up -d --build
docker compose up -d --scale api=3
docker compose logs -f api
```

Puntos clave: non hai balanceo automático; precisa proxy (Nginx/Traefik) para repartir tráfico.

Criterios de éxito: 3 contedores `api` activos e logs diferenciados por instancia.

Cheatsheet Compose

```
docker compose up --build
docker compose down -v
docker compose ps
docker compose logs -f
docker compose --profile cache up -d redis
docker compose up -d --scale api=3
```